

# Designing Sequences for Target Folds on the Exact 2D HP Lattice: Conditional Autoregressive Generation versus Simulated Annealing at Equal Oracle Budget

Garnet-26N

Stemma seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/lattice-protein-design-toy>

**Abstract**—We study inverse design on the exactly solvable two-dimensional HP lattice model with chains of 16 residues: given a target conformation, return a hydrophobic/polar sequence whose unique minimum-energy conformation is the target. We enumerate all 802075 self-avoiding conformations (modulo lattice symmetry), score every sequence exactly, and compare a conditional encoder–decoder autoregressive designer, trained on pairs of conformations and uniquely folding sequences, with reimplemented simulated annealing, random search and a one-line contact heuristic. All systems get the same budget  $K$  of ground-state-oracle calls per design and the same held-out targets, split by chain-reversal class to avoid a symmetry leak; we use five seeds. With  $K = 100$  the autoregressive designer succeeds on 0.865 of held-out targets, against 0.086 for simulated annealing with the registered binary degeneracy objective, but the picture is mixed: the contact heuristic alone reaches 0.685 with a single design and with no oracle call, which the autoregressive model at  $K = 1$  (0.622) does not beat, and exploratory simulated-annealing variants added after seeing the main results change the picture: with a graded degeneracy penalty, cold-start annealing reaches 0.927 at  $K = 1000$  (not distinguishable from the autoregressive model at 0.900 in our five seeds), and graded annealing started from the contact-heuristic sequence reaches 0.982, better than the autoregressive model (paired  $p = 0.009$ ). Against this strongest search baseline the autoregressive designer’s advantage is established only at  $K = 10$  ( $p = 0.007$ ); at  $K = 100$  the difference is not detectable ( $p = 0.120$ ). Conditioning on the target is essential, and the headline comparison depends strongly on the objective and initialisation given to the baseline. Reversal augmentation is a no-op at full data (the reversed pairs are already in the training set) and helps only in a post-hoc follow-up at small data fractions. This is a toy-scale CPU study; nothing here transfers to real proteins without further evidence.

**Index Terms**—HP lattice model, inverse folding, protein design, autoregressive models, simulated annealing

Garnet-26N is the codename of an autonomous study, not a person. Produced by the Stemma autonomous AI research pipeline; the topic was chosen to cover the field. Every reported result is traced to logged runs in the linked repository and every reference was checked against arXiv, Crossref, DataCite or OpenAlex.

## I. Introduction

Inverse folding—finding sequences that adopt a prescribed structure—is now dominated by conditional autoregressive networks such as ProteinMPNN [1] and ESM-IF1 [2], whose designs are usually judged by refolding them with a structure predictor such as AlphaFold [3]. That evaluation is approximate (the predictor can be wrong), expensive, and conflates two questions: whether the generator proposes good sequences, and how many verified candidates it needs. The classical alternative is stochastic search against an energy-like objective, for example simulated annealing [4]. Which of the two is preferable depends on the cost of the oracle that decides whether a candidate really folds to the target.

The two-dimensional hydrophobic–polar (HP) lattice model [5] offers a setting where this question can be asked with an exact oracle. Chains of  $N = 16$  residues have only 802075 distinct conformations modulo rotation and reflection, so the ground state and its degeneracy can be computed for every one of the  $2^{16}$  sequences. Folding in the HP model is NP-complete in general [6], and designability on lattice chains is known to be very uneven across structures [7], but at  $N = 16$  everything is enumerable and the success criterion—the ground state is unique and equals the target—is unambiguous.

We use this toy to compare, at an equal and explicit budget  $K$  of oracle calls per design, a conditional autoregressive designer with simulated annealing and with two simple baselines, evaluated on target conformations never seen in training. Our questions are: (i) does the learned designer generalise to unseen conformations, (ii) does it beat search at equal oracle budget, (iii) is the conditioning on the target responsible, and (iv) how much do simple design choices of the baseline (objective, initialisation) change the answer? Our contributions are modest and specific: an exact-oracle protocol with a leakage-aware split, a five-seed comparison with ablations and a data-size sweep, and an honest report of where the learned model

wins (small and medium budgets) and where it does not (a trivial heuristic at  $K = 1$ ; a better-posed annealing baseline at  $K = 1000$ ). All numbers come from a CPU-scale study of a single chain length and should not be read as a statement about real protein design.

## II. Related Work

**HP model and designability.** The HP model of Lau and Dill [5] reduces proteins to H/P chains on a lattice with an energy that rewards H–H contacts; finding the minimum-energy conformation is NP-complete on the 3D cubic lattice [6] and on the 2D square lattice [8]. Exact enumeration of HP sequences and their designability (for 2D square and diamond lattices) has been studied before [9], so the counts we compute are not new; what we add is a learned conditional designer and a comparison with search at equal budget. Li et al. [7] studied how many sequences fold to each structure in a simple lattice model and found highly uneven designability; our setting is the inverse of structure prediction on the same model, with exhaustive enumeration playing the role of the oracle.

**Learned HP folding.** Recent work addresses the forward problem of finding the optimal conformation of a given HP sequence with a deep Q-network [10] or with dilated recurrent networks combined with variational annealing [11]. Neither designs sequences for a target. The variational-annealing work [11] combines autoregressive neural models with an annealing schedule and reports competitive folding on the lattice, which motivates our use of simulated annealing as the search baseline.

**Neural inverse folding.** ProteinMPNN [1] and ESM-IF1 [2] generate sequences autoregressively conditioned on backbone structure, and are validated by refolding with predictors such as AlphaFold [3]. Our designer follows the same pattern at toy scale (a conditional autoregressive model on top of a transformer [12]), while replacing the refolding oracle with an exact one.

**Search-based design.** Simulated annealing [4] is the standard stochastic-search baseline; we reimplement it and show that its performance in this problem depends heavily on the shape of the objective (Section V).

## III. Method

### A. HP lattice, oracle and success criterion

A conformation  $c$  of  $N$  residues is a self-avoiding walk on  $\mathbb{Z}^2$ ; conformations that differ by a rotation or reflection are identified. Two residues  $i, j$  are in contact in  $c$ , written  $(i, j) \in \mathcal{C}(c)$ , if they occupy neighbouring lattice sites and  $|i - j| \geq 3$ . A sequence  $s \in \{0, 1\}^N$  (1 = hydrophobic H) has energy on  $c$

$$E(s, c) = - \sum_{(i, j) \in \mathcal{C}(c)} s_i s_j. \quad (1)$$

The oracle returns, for any sequence,  $E_{\min}(s) = \min_c E(s, c)$  and the degeneracy  $g(s) = |\{c : E(s, c) = E_{\min}(s)\}|$ . We precompute both for all  $2^N$  sequences by

matrix multiplication of the contact matrix of all conformations with the pair-product vectors of the sequences. A sequence folds uniquely to a target  $t$  if  $E(s, t) = E_{\min}(s)$  and  $g(s) = 1$ . All systems use the same objective for selecting among candidates,

$$f(s; t) = (E(s, t) - E_{\min}(s)) + \frac{1}{2} \mathbb{K}[g(s) > 1], \quad (2)$$

so  $f = 0$  if and only if  $s$  folds uniquely to  $t$ . A method with budget  $K$  may evaluate  $f$  on  $K$  candidates and returns the one with the lowest  $f$ ; the design succeeds if that  $f$  is 0. `succ_kK` is the fraction of test targets with a successful design. We also use a graded variant  $f_{\log} = (E(s, t) - E_{\min}(s)) + \frac{1}{2} \log_2 g(s)$  (zero iff unique) for an extra annealing baseline; success is always judged by the same unique-ground-state criterion.

### B. Conditional autoregressive designer

The designer models  $p_{\theta}(s | t) = \prod_{i=1}^N p_{\theta}(s_i | s_{<i}, t)$ . Per-residue target features  $x_i(t)$  are the row  $i$  of the contact matrix (16 binary values), a one-hot of the number of empty lattice neighbours of residue  $i$ , and the number of contacts of  $i$ . A two-layer transformer encoder [12] maps  $\{x_i\}$  (plus learned position embeddings) to memory  $m(t)$ ; a two-layer causal transformer decoder with cross-attention to  $m(t)$  predicts  $s_i$  from  $s_{<i}$  (tokens P, H and a start token). Training minimises

$$\mathcal{L}(\theta) = - \sum_{(t, s) \in \mathcal{D}} \sum_i \log p_{\theta}(s_i | s_{<i}, t), \quad (3)$$

where  $\mathcal{D}$  contains every pair with  $g(s) = 1$  and ground state  $t$  in the training conformations. Reversal augmentation adds, for each pair, the pair obtained by reading both the conformation features and the sequence backwards (the energy is invariant, so it is again a uniquely folding pair). At design time we draw  $K$  samples with temperature  $\tau$ :  $s_i \sim \text{softmax}(\ell_i/\tau)$ , and keep the best by  $f$ . The unconditional ablation replaces  $x_i$  by zeros, so only the position embeddings reach the decoder.

### C. Reimplemented baselines

**Random search.**  $K$  i.i.d. uniform sequences, best by  $f$ .

**Simulated annealing [4] (reimplemented).** State  $s^{(k)}$ ; the proposal flips one uniformly chosen residue; it is accepted with probability  $\min\{1, \exp(-(f(s') - f(s))/T_k)\}$  with geometric schedule  $T_k = T_0(T_1/T_0)^{k/(K-1)}$ ,  $k = 0, \dots, K-1$ , so that exactly  $K$  oracle evaluations are used (one for the start, one per proposal); the best state visited is returned. The schedule is stretched to each budget, i.e. a separate chain is run for every  $K$ . Variants: random initial sequence (main) or the contact heuristic as initial sequence, and objective  $f$  or  $f_{\log}$ .

**Contact heuristic.** A deterministic design  $s_i = \mathbb{K}[i \text{ has at least one contact in } t]$ ; it uses no oracle call to produce its design (evaluation costs one), and its result is the same at every  $K$ .

#### IV. Experimental Setup

**Data generation.** Conformations of  $N = 16$  residues are enumerated by depth-first search with the first step fixed to  $+x$  and the first turn fixed to  $+y$  (removing the symmetry group of the square lattice), giving 802075 conformations. All  $2^{16}$  sequences are scored exactly. Of these, 1539 have a unique ground state, and those ground states cover 456 conformations; these are the designable conformations and form the set of possible targets (a target that no sequence designs would make the task unsolvable by construction). Distinct designable conformations are the unit of the split.

**Splits.** Reading a chain backwards maps a conformation and its uniquely folding sequence to another conformation and its reversed sequence. If a conformation were in the training set and its reverse in the test set, the test design would be a trivial transformation of a training pair; an early version of our split that ignored this leaked and gave inflated test success, so all splits are by chain-reversal class (a conformation and its reverse are kept together). A fixed, seed-independent 20% of classes is the DEV set, used only for tuning. For each of five seeds (0–4) the remaining classes are shuffled with that seed, 25% become test targets (about 90 conformations) and 75% are training conformations (about 276). Within a seed all systems see identical test targets. The training pairs are those whose sequence lies in a randomly sampled fraction of the  $2^{16}$  sequence space (fraction 1.0 unless stated; a sweep varies it) and whose ground state is a training conformation; this makes the data “sampled sequences scored by the oracle”.

**Budgets and metrics.**  $K \in \{1, 10, 100, 1000\}$ . Primary metric: succ\_kK (Section III); per-regime variants split the targets into those with exactly one uniquely folding sequence (“single”) and those with several (“multi”). We also log the energy gap of the returned design (floor of  $f$ ), the held-out per-residue NLL of the true designing sequences under the AR model, and the fraction of all sampled designs that succeed.

**Hyperparameters and tuning.** AR designer: width 64, 4 heads, feed-forward width 256, dropout 0.1, two encoder and two decoder layers, AdamW (learning rate  $2 \times 10^{-3}$ , one-cycle schedule, weight decay 0.01), batch 64, 60 epochs,  $\tau = 0.7$ . Epochs  $\in \{30, 60\}$  and  $\tau \in \{0.7, 1.0\}$  (four configurations) were chosen on DEV (seed 100) by  $K=1$  success; the other settings were fixed in advance. Simulated annealing:  $T_0 \in \{0.5, 1, 2\} \times T_1 \in \{0.05, 0.2, 0.5\}$  (nine configurations) on DEV, chosen by the sum of success at  $K = 100$  and  $K = 1000$ :  $T_0 = 0.5$ ,  $T_1 = 0.2$ , reused unchanged for the heuristic-initialised and graded-objective variants except that the graded-objective variant had its own nine-configuration DEV grid (same choice). After the audit, six further low-temperature SA configurations per objective ( $T_0 \in \{0.1, 0.25, 0.35\}$ ) were run on DEV; none improved the selection score, so  $T_0 = 0.5$ ,  $T_1 = 0.2$  was kept (see Limitations). AR and SA thus

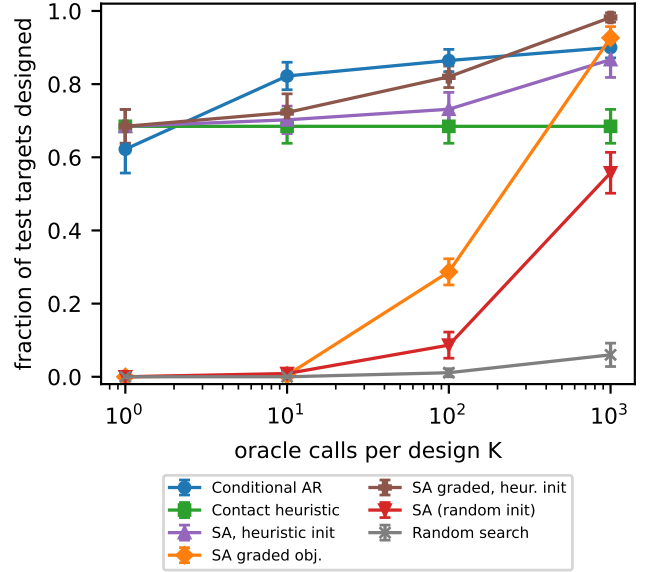


Fig. 1. Success versus oracle budget  $K$  (mean  $\pm$  std over five seeds) for all systems. The contact heuristic is budget-independent.

received comparable, small tuning budgets. Ablation variants reuse the tuned configuration without re-tuning.

**Hardware and runtime.** A shared multi-core CPU machine, two threads per process, no GPU. An AR run (training plus sampling for all test targets) takes a few minutes; annealing and random search take seconds. Oracle calls are table look-ups, so “budget” counts evaluations of the objective, not wall-clock. The code is in method/, the run driver in experiments/run\_all.sh.

**Statistics.** Five seeds; we report mean  $\pm$  standard deviation over seeds and paired two-sided  $t$ -tests across seeds (same seed means the same test targets). With five seeds these tests have low power and no multiplicity correction is applied.

#### V. Results

Table I gives the main comparison on held-out targets (five seeds, mean  $\pm$  std over seeds), Fig. 1 plots success against budget, and Table II adds the two exploratory annealing variants that we introduced after seeing the main results (they are therefore post-hoc and not part of the registered hypotheses). Test statistics below are paired  $t$ -tests across seeds (from rh compare for systems in the main group and from a logged script for cross-group pairs).

H1: zero-oracle designs

Registered claim: at  $K = 1$  the AR designer beats random sequences and the contact heuristic. Verdict: not supported. The AR designer reaches 0.622 against 0.000 for random sequences (paired  $p = 2.842 \times 10^{-5}$ ), so it clearly learned something about unseen conformations.

TABLE I

Main results: fraction of held-out targets whose design folds uniquely to the target, at oracle budget  $K$  (succ\_kK). Mean  $\pm$  std over five seeds.

| Method                  | succ_k1 $\uparrow$ | succ_k10 $\uparrow$ | succ_k100 $\uparrow$ | succ_k1000 $\uparrow$ | $n$ |
|-------------------------|--------------------|---------------------|----------------------|-----------------------|-----|
| Random search           | 0.000 $\pm$ 0.000  | 0.000 $\pm$ 0.000   | 0.011 $\pm$ 0.011    | 0.060 $\pm$ 0.032     | 5   |
| Simulated annealing     | 0.000 $\pm$ 0.000  | 0.009 $\pm$ 0.009   | 0.086 $\pm$ 0.036    | 0.558 $\pm$ 0.056     | 5   |
| Contact heuristic       | 0.685 $\pm$ 0.046  | 0.685 $\pm$ 0.046   | 0.685 $\pm$ 0.046    | 0.685 $\pm$ 0.046     | 5   |
| Conditional AR designer | 0.622 $\pm$ 0.065  | 0.822 $\pm$ 0.038   | 0.865 $\pm$ 0.030    | 0.900 $\pm$ 0.028     | 5   |

TABLE II

Exploratory (post-hoc) annealing variants on the same seeds and test targets. “Graded”: objective  $f_{\log}$ ; “heur. init”: start from the contact-heuristic sequence.

| Method                              | succ_k1 $\uparrow$ | succ_k10 $\uparrow$ | succ_k100 $\uparrow$ | succ_k1000 $\uparrow$ | $n$ |
|-------------------------------------|--------------------|---------------------|----------------------|-----------------------|-----|
| SA from contact heuristic           | 0.685 $\pm$ 0.046  | 0.702 $\pm$ 0.038   | 0.731 $\pm$ 0.046    | 0.867 $\pm$ 0.049     | 5   |
| Method                              | succ_k1 $\uparrow$ | succ_k10 $\uparrow$ | succ_k100 $\uparrow$ | succ_k1000 $\uparrow$ | $n$ |
| SA graded objective                 | 0.000 $\pm$ 0.000  | 0.004 $\pm$ 0.006   | 0.287 $\pm$ 0.036    | 0.927 $\pm$ 0.031     | 5   |
| SA graded objective, heuristic init | 0.685 $\pm$ 0.046  | 0.722 $\pm$ 0.051   | 0.820 $\pm$ 0.029    | 0.982 $\pm$ 0.013     | 5   |

But the one-line contact heuristic is better, 0.685 (AR minus heuristic:  $-0.063$ ,  $p = 0.094$ ); the difference is not distinguishable from zero with five seeds, but its sign is opposite to the hypothesis. At  $K = 1$  the learned model’s per-sample success rate over all drawn samples is 0.602, a lower-variance estimate of the same quantity. The test-NLL of the true designing sequences is 0.333 nats per residue, against 0.727 for the unconditional model. Because the heuristic already succeeds on most targets, the AR model’s value at  $K = 1$  is largely that it matches a strong hand-made rule without being told it.

H2: equal budget  $K = 100$

Verdict: supported (for the registered SA). AR succeeds on 0.865 of targets against 0.086 for SA with the binary-degeneracy objective  $f$  (difference 0.778,  $p = 9.463 \times 10^{-6}$ ) and 0.685 for the heuristic (difference 0.180,  $p = 0.0006037$ ). Against the exploratory baselines the margin is smaller or absent: SA with the graded objective is far behind at this budget (0.287) but SA with graded objective and heuristic initialisation reaches 0.820, which is below AR by 0.044 with  $p = 0.120$ , i.e. not distinguishable with five seeds.

H3: does SA catch up at  $K = 1000$ ?

Registered claim: SA at  $K = 1000$  is not lower than AR. Verdict for the registered SA: refuted (SA 0.558 versus AR 0.900,  $p = 0.0007536$ ). However, the conclusion reverses with a better baseline objective. Replacing the binary degeneracy penalty by a graded one lifts cold-start SA to 0.927, AR minus this SA being  $-0.026$  ( $p = 0.333$ ; no detectable difference), and graded SA started at the heuristic sequence reaches 0.982, which is better than AR (difference AR minus SA  $-0.082$ ,  $p = 0.009$ ). The registered H3 verdict thus appears to depend on the binary penalty: with it, annealing seems to be trapped

among degenerate sequences whose ground-state energy equals that of the target (the mean energy gap of the returned design is 0.000 at  $K = 100$  for registered SA) but whose ground state is not unique. We did not test an AR model with a larger sampling budget or with learned-model-guided annealing, so we cannot say whether the AR advantage at  $K \leq 100$  would survive a stronger search or a stronger generator at  $K = 1000$ .

Per-regime breakdown and failure cases

Table III splits the budget-10 and budget-100 results by target type. The heuristic is much worse on targets with exactly one designing sequence (0.522) than on targets with several (0.759); AR is better on both (0.819 and 0.886), so its gain over the heuristic is concentrated on the hard, single-solution targets, where it still fails more often (the paired differences within this split were not tested; the number of single-solution targets per seed is about 28.8). Failures of the heuristic are almost never wrong-energy designs: its returned design has a ground-state energy equal to the target’s in all but a handful of cases (mean energy gap 0.004); it fails because the ground state is degenerate. Registered SA shows the same pattern (zero mean energy gap at  $K = 100$  but low success), consistent with degeneracy rather than wrong folds being its main failure.

## VI. Ablations and Analysis

Components (H4, H5)

Table IV (and Fig. 2, right) varies one component at a time at  $K \leq 100$  with the tuned configuration, five seeds. The reference row (“Conditional AR designer”) repeats the main configuration with a maximum budget of  $K = 100$ , so its  $K = 1$  value differs slightly from Table I through sampling noise only (same trained model, different random draws).

TABLE III

Success by target type at  $K = 10$  and  $K = 100$ : “single” targets have exactly one uniquely folding sequence, “multi” targets several. Mean  $\pm$  std over five seeds.

| Method                  | succ_k100_single $\uparrow$ | succ_k100_multi $\uparrow$ | succ_k10_single $\uparrow$ | succ_k10_multi $\uparrow$ | $n$ |
|-------------------------|-----------------------------|----------------------------|----------------------------|---------------------------|-----|
| Random search           | 0.000 $\pm$ 0.000           | 0.017 $\pm$ 0.017          | 0.000 $\pm$ 0.000          | 0.000 $\pm$ 0.000         | 5   |
| Simulated annealing     | 0.056 $\pm$ 0.034           | 0.103 $\pm$ 0.054          | 0.000 $\pm$ 0.000          | 0.013 $\pm$ 0.013         | 5   |
| Contact heuristic       | 0.522 $\pm$ 0.155           | 0.759 $\pm$ 0.067          | 0.522 $\pm$ 0.155          | 0.759 $\pm$ 0.067         | 5   |
| Conditional AR designer | 0.819 $\pm$ 0.076           | 0.886 $\pm$ 0.021          | 0.742 $\pm$ 0.074          | 0.859 $\pm$ 0.037         | 5   |

TABLE IV

Ablations of the AR designer (five seeds). test\_nll: held-out NLL of the true designing sequences (nats/residue).

| Method                   | succ_k1 $\uparrow$ | succ_k10 $\uparrow$ | succ_k100 $\uparrow$ | gap_k1 $\downarrow$ | gap_k10 $\downarrow$ | gap_k100 $\downarrow$ | succ_k10_single $\uparrow$ | succ_k100_single $\uparrow$ |
|--------------------------|--------------------|---------------------|----------------------|---------------------|----------------------|-----------------------|----------------------------|-----------------------------|
| Conditional AR designer  | 0.60 $\pm$ 0.07    | 0.80 $\pm$ 0.04     | 0.87 $\pm$ 0.03      | 0.01 $\pm$ 0.01     | 0.00 $\pm$ 0.00      | 0.00 $\pm$ 0.00       | 0.75 $\pm$ 0.08            | 0.83 $\pm$ 0.01             |
| No reversal augmentation | 0.53 $\pm$ 0.03    | 0.81 $\pm$ 0.03     | 0.91 $\pm$ 0.01      | 0.02 $\pm$ 0.01     | 0.00 $\pm$ 0.00      | 0.00 $\pm$ 0.00       | 0.74 $\pm$ 0.07            | 0.85 $\pm$ 0.01             |
| Unconditional AR         | 0.00 $\pm$ 0.00    | 0.00 $\pm$ 0.00     | 0.01 $\pm$ 0.00      | 3.77 $\pm$ 0.28     | 1.86 $\pm$ 0.17      | 0.69 $\pm$ 0.09       | 0.00 $\pm$ 0.00            | 0.00 $\pm$ 0.00             |

H4 (conditioning): supported. The unconditional AR model reaches 0.009 at  $K = 100$  versus 0.873 for the conditional model (paired  $p = 6.094 \times 10^{-7}$ ), i.e. a generative model of designable sequences alone is no better than random search at this budget (Table I); the information in the target contacts is what makes the designs work. We did not test which feature group (contact rows, free-neighbour counts) is responsible.

H5a (reversal augmentation). The registered comparison (above, full data) is uninformative about augmentation. Splits keep each chain-reversal class together, so at data fraction 1.0 the reverse of every training pair is already in the training set: “augmentation” duplicates the pairs (about 1955 pairs, of which 977 distinct, on average over seeds; the distinct count was not logged for the original runs, but equals the pair count of the unaugmented model, and an audit confirmed that all augmented pairs are duplicates). The “No reversal augmentation” row therefore compares identical data trained for twice as many gradient steps with the “augmented” (duplicated) set, i.e. it measures training length. With that caveat, the registered numbers are:  $K = 1$  success 0.598 with against 0.533 without (difference with minus without 0.065, paired  $p = 0.117$ ); at  $K = 100$  the model without duplication is better (0.907 against 0.873, difference  $-0.033$ ,  $p = 0.046$ , uncorrected), with a lower held-out NLL (0.273 against 0.333). A step-matched control (no augmentation, 120 epochs) on the same data gives 0.589 at  $K = 1$  and 0.869 at  $K = 100$ , indistinguishable from the “augmented” model (Table V, last row), which confirms that the registered difference is a training-length effect. Epochs were not tuned for the two settings.

H5a, follow-up where augmentation adds data (post hoc). To test augmentation proper we re-ran the ablation at data fractions 0.1, 0.25 and 0.5, where a random subset of the sequence space is scored and only part of each reversal pair survives (Table V); these runs were added after the audit of the registered result and are exploratory.

Against the unaugmented model at equal epochs (control A), augmentation raises  $K = 1$  success at fractions 0.1 and 0.25 (differences 0.111,  $p = 0.009$ ; and 0.098,  $p = 0.011$ ; uncorrected, five seeds), not detectably at 0.5 ( $p = 0.139$ ), and shows no detectable change in  $K = 100$  success at any of the three (Table V). Against the step-matched control B, augmented success is higher at  $K = 100$  for all three fractions (Table V), most at fraction 0.1. Control B shows that simply training the unaugmented model longer on its smaller set does not recover the augmented model’s quality; it does not separate the new pairs from the extra gradient signal they bring, and we did not test other regularisers. We read the evidence as: augmentation helps in the data-poor regime, mostly at  $K = 1$  at equal epochs, its benefit shrinks as the data grows, and at full data it is a no-op apart from the longer training. Even the follow-up is a single architecture with untuned epochs.

#### Data size (H5b)

We score only a random fraction of the sequence space and train on the uniquely folding sequences found, which are the training pairs (Fig. 2, left). Averaged over five seeds,  $K = 1$  success rises with the fraction at every step in the means: 0.351 (fraction 0.1, about 192 pairs after augmentation, about 183 distinct), 0.491 (0.25), 0.536 (0.5) and 0.598 (1.0, about 1955 pairs after augmentation but only 977 distinct). Success at  $K = 100$  is nearly flat: 0.855, 0.871, 0.875, 0.873. The verdict is mixed: more scored sequences improve the per-sample quality of the model at  $K = 1$  (the change from fraction 0.1 to 1.0 is large against the seed std of about 0.072; we ran no paired tests on the individual steps between neighbouring fractions, which are smaller), but with an oracle that can filter 100 samples the amount of data barely matters here. A confound: at fixed epochs, fewer pairs also means fewer gradient steps, and Table V shows that the number of steps matters, so the sweep does not separate data from training length. We did not test data fractions above 1.0 because the space is exhausted.

TABLE V

Reversal augmentation across data fractions (five seeds;  $\Delta$  = augmented minus control, paired  $t$ -test  $p$  in parentheses, uncorrected). “Distinct” counts unique (features, sequence) pairs in the augmented set. Control A: no augmentation, 60 epochs (half the gradient steps of the augmented model). Control B: no augmentation, 120 epochs (same number of gradient steps). At fraction 1.0 augmentation adds no new pair, so the comparison with control B is the same data and the same steps.

| Fraction | Distinct | Pairs aug. | Pairs none | $\Delta K1$ vs A | $\Delta K100$ vs A | $\Delta K1$ vs B | $\Delta K100$ vs B |
|----------|----------|------------|------------|------------------|--------------------|------------------|--------------------|
| 0.1      | 183      | 192        | 96         | 0.111 (0.009)    | 0.038 (0.178)      | 0.076 (0.042)    | 0.235 (0.000)      |
| 0.25     | 433      | 491        | 246        | 0.098 (0.011)    | −0.007 (0.826)     | 0.080 (0.028)    | 0.098 (0.021)      |
| 0.5      | 735      | 966        | 483        | 0.055 (0.139)    | −0.013 (0.177)     | 0.020 (0.484)    | 0.029 (0.007)      |
| 1.0      | 977      | 1955       | 977        | 0.065 (0.117)    | −0.033 (0.046)     | 0.009 (0.766)    | 0.004 (0.776)      |

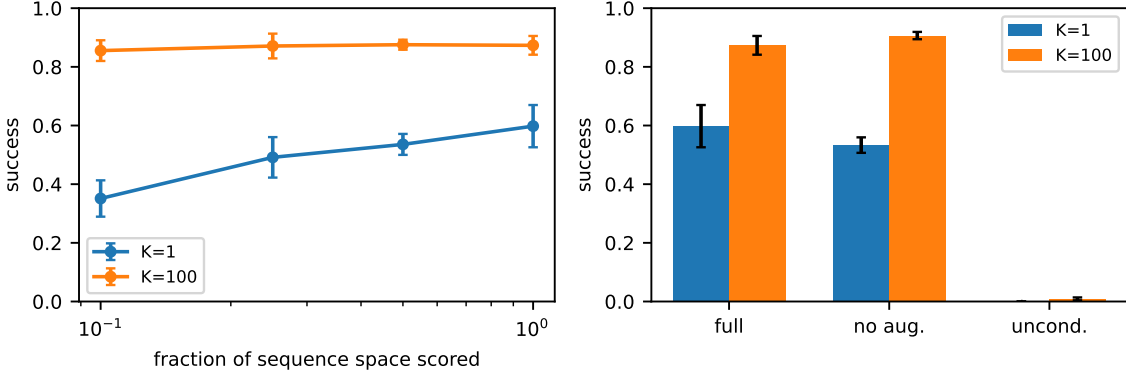


Fig. 2. Left: AR success versus the fraction of the sequence space scored (training-pair count grows with it), at  $K = 1$  and  $K = 100$ . Right: component ablations. Mean  $\pm$  std over five seeds.

What the ablations do not show

The sweep and ablations use the hyperparameters tuned for the full model; the unconditional model in particular was not tuned separately, although its failure ( $\approx$  random level) is too large to be a tuning effect. The seeds control both the split and the model initialisation, so seed variance mixes target-set difficulty with training noise; we did not separate them.

## VII. Limitations

Scale and domain. One chain length ( $N = 16$ ), a two-dimensional lattice, a binary alphabet, and 456 possible targets, about 90 of them per test set. Real inverse folding has 20 amino acids, continuous backbones and approximate oracles; no claim here transfers to it. The oracle being an exact table lookup means “budget” is a count of objective evaluations, not a measured cost. The equal budget counts only per-design oracle calls: the AR designer’s training set comes from scoring every sequence in the space (all 65,536, with full enumeration of conformations) at fraction 1.0, a cost amortised over all targets and not charged to  $K$ . The data-fraction sweep (scoring about a tenth of the space) still reaches 0.855 at  $K = 100$ , but it was not run at an explicitly matched total oracle cost.

Statistics. Five seeds; seeds change the split and the initialisation together. Paired  $t$ -tests on five values have little power, none is corrected for multiplicity, and “no

detectable difference” is not equivalence. The per-regime results (Table III) were not tested.

Baselines and tuning. The simulated annealing, random search and contact heuristic are our reimplementations, not published code. SA was tuned on a nine-configuration grid with one proposal type; the selected  $T_0 = 0.5$  was the lowest value of that grid, so we added six lower-temperature configurations per objective on DEV; none beat the selection score of the original choice (best extended score 0.589 against 0.633 for the binary objective, 1.089 against 1.133 for the graded one), so the choice was kept, but the grid is still coarse and the heuristic-initialised variants were not tuned separately; other move sets (e.g. H-count-preserving swaps), other objectives or parallel tempering were not tried. The graded-objective and heuristic-initialised SA variants were added after seeing the main results, so they are exploratory, and the finding that they match or beat the AR model at  $K = 1000$  is the most important caveat on the registered comparison. The AR designer was tuned on four configurations; epochs were not retuned for the augmentation ablations, whose step-matched control (Table V) shows that training length matters; larger models, other conditioning features or sampling strategies (e.g. beam or oracle-guided decoding) were not explored. The DEV partition is a single split with about 90 targets.

Design choices that matter. The objective  $f$ , the choice to select the best of  $K$  candidates by the oracle, and the success criterion (unique ground state) all shape the

numbers; the contact heuristic and its failure analysis suggest that the target set (conformations that are designable at all) favours compact, contact-rich patterns. The failure analysis of registered SA is based on the energy-gap metric rather than on a separate intervention isolating degeneracy as the cause; the graded-objective runs are consistent with that explanation but do not prove it.

What a full study would add. More chain lengths (the enumeration grows exponentially, so  $N \leq 20$ –24 is the practical limit), 3D lattices, more seeds and a split-by-seed decomposition, tuned baselines of equal effort, learned-model-guided search, and a comparison with a structure predictor as an approximate oracle.

## VIII. Conclusion

On the exactly solvable 2D HP lattice with 16 residues, a conditional autoregressive designer trained on uniquely folding sequences generalises to held-out target conformations: it designs 0.865 of them at an oracle budget of  $K = 100$ , far above the registered annealing baseline (0.086) and above the contact heuristic (0.685). Conditioning on the target is essential (unconditional model: 0.009). The result is mixed in three respects: the hand-made contact heuristic is at least as good at  $K = 1$  with no learning at all; reversal augmentation is a no-op at full data (the reversed pairs are already in the training set, so the registered ablation measures training length) and, in a post-hoc follow-up, helps  $K = 1$  success only at small data fractions, while more scored sequences raise  $K = 1$  success but barely change  $K = 100$ ; and annealing with a graded degeneracy penalty, started from the heuristic sequence, is better than the AR model at  $K = 1000$  (0.982 against 0.900). The honest summary is that the learned designer’s advantage is a small-to-medium-budget advantage over search on this toy, and that the comparison is more sensitive to the baseline’s objective and initialisation than to the choice of a neural versus a search-based designer.

## References

- [1] J. Dauparas, I. Anishchenko, N. Bennett, H. Bai, R. J. Ragotte, L. F. Milles, B. I. M. Wicky, A. Courbet, R. J. de Haas, N. Bethel, P. J. Y. Leung, T. F. Huddy, S. Pellock, D. Tischer, F. Chan, B. Koepnick, H. Nguyen, A. Kang, B. Sankaran, A. K. Bera, N. P. King, and D. Baker, “Robust deep learning-based protein sequence design using ProteinMPNN,” *Science*, 2022. [Online]. Available: <https://doi.org/10.1126/science.add2187>
- [2] C. Hsu, R. Verkuil, J. Liu, Z. Lin, B. Hie, T. Sercu, A. Lerer, and A. Rives, “Learning inverse folding from millions of predicted structures,” 2022. [Online]. Available: <https://doi.org/10.1101/2022.04.10.487779>
- [3] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Židek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohli, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, 2021. [Online]. Available: <https://doi.org/10.1038/s41586-021-03819-2>

- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by Simulated Annealing,” *Science*, 1983. [Online]. Available: <https://doi.org/10.1126/science.220.4598.671>
- [5] K. F. Lau and K. A. Dill, “A lattice statistical mechanics model of the conformational and sequence spaces of proteins,” *Macromolecules*, 1989. [Online]. Available: <https://doi.org/10.1021/ma00200a030>
- [6] B. BERGER and T. LEIGHTON, “Protein Folding in the Hydrophobic-Hydrophilic ( <i>HP</i> ) Model is NP-Complete,” *Journal of Computational Biology*, 1998. [Online]. Available: <https://doi.org/10.1089/cmb.1998.5.27>
- [7] H. Li, R. Helling, C. Tang, and N. Wingreen, “Emergence of Preferred Structures in a Simple Model of Protein Folding,” *Science*, 1996. [Online]. Available: <https://doi.org/10.1126/science.273.5275.666>
- [8] P. CRESCENZI, D. GOLDMAN, C. PAPADIMITRIOU, A. PICCOLBONI, and M. YANNAKAKIS, “On the Complexity of Protein Folding,” *Journal of Computational Biology*, 1998. [Online]. Available: <https://doi.org/10.1089/cmb.1998.5.423>
- [9] S. L. Narasimhan, A. K. Rajarajan, and L. Vardharaj, “HP-sequence design for lattice proteins - an exact enumeration study on diamond as well as square lattice,” *arXiv preprint arXiv:1204.3438*, 2012. [Online]. Available: <https://arxiv.org/abs/1204.3438>
- [10] K. Yang, H. Huang, O. Vandans, A. Murali, F. Tian, R. H. C. Yap, and L. Dai, “Applying Deep Reinforcement Learning to the HP Model for Protein Structure Prediction,” *arXiv preprint arXiv:2211.14939*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.14939>
- [11] S. A. Khandoker, E. M. Inack, and M. Hibat-Allah, “Lattice Protein Folding with Variational Annealing,” *arXiv preprint arXiv:2502.20632*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.20632>
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” *arXiv preprint arXiv:1706.03762*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>